

老子软考，扫码下载



老子
软考

老子软考
软考用老子

软件设计师-2015年下半年-选择题

第1题：CPU是在（1）结束时响应DMA请求的。

- A. 一条指令执行
- B. 一段程序
- C. 一个时钟周期
- D. 一个总线周期

第2题：虚拟存储体系由（2）两级存储器构成。

- A. 主存-辅存
- B. 寄存器-Cache
- C. 寄存器-主存
- D. Cache-主存

第3题：浮点数能够表示的数的范围是由其（3）的位数决定的。

- A. 尾数
- B. 阶码
- C. 数符
- D. 阶符

第4题：在机器指令的地址字段中，直接指出操作数本身的寻址方式称为（4）。

- A. 隐含寻址
- B. 寄存器寻址
- C. 立即寻址
- D. 直接寻址

第5题：内存按字节编址从B3000H到DABFFH的区域其存储容量为（5）。

- A. 123KB
- B. 159KB
- C. 163KB
- D. 194KB

第6题：CISC是（6）的简称。

- A. 复杂指令系统计算机
- B. 超大规模集成电路
- C. 精简指令系统计算机
- D. 超长指令字

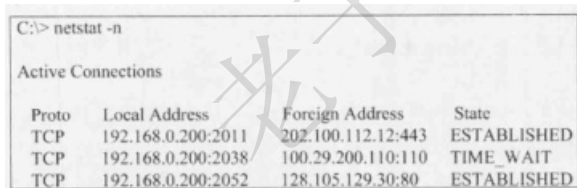
第7题：（7）不属于主动攻击。

- A. 流量分析
- B. 重放
- C. IP地址欺骗
- D. 拒绝服务

第8题：防火墙不具备（8）功能。

- A. 记录访问过程
- B. 查毒
- C. 包过滤
- D. 代理

第9题：根据下图所示的输出信息，可以确定的是：（9）



Proto	Local Address	Foreign Address	State
TCP	192.168.0.200:2011	202.100.112.12:443	ESTABLISHED
TCP	192.168.0.200:2038	100.29.200.110:110	TIME_WAIT
TCP	192.168.0.200:2052	128.105.129.30:80	ESTABLISHED

- A. 本地主机正在使用的端口号是公共端口号
- B. 192.168.0.200正在与128.105.129.30建立连接
- C. 本地主机与202.100.112.12建立了安全连接
- D. 本地主机正在与100.29.200.110建立连接

第10题：以下著作权权利中，（10）的保护期受时间限制。

- A. 署名权

- B. 修改权
- C. 发表权
- D. 保护作品完整权

第11题：王某在其公司独立承担了某综合信息管理系统软件的程序设计工作。该系统交付用户、投入试运行后，王某辞职，并带走了该综合信息管理系统源程序，拒不交还公司。王某认为，综合信息管理系统源程序是他独立完成的：他是综合信息管理系统源程序的软件著作权人。王某的行为（11）。

- A. 侵犯了公司的软件著作权
- B. 未侵犯公司的软件著作权
- C. 侵犯了公司的商业秘密权
- D. 不涉及侵犯公司的软件著作权

第12题：声音（音频）信号的一个基本参数是频率，它是指声波每秒钟变化的次数，用Hz表示。人耳能听到的音频信号的频率范围是（12）。

- A. 0Hz~20KHz
- B. 0Hz~200KHz
- C. 20Hz~20KHz
- D. 20Hz~200KHz

第13题：颜色深度是表达图像中单个像素的颜色或灰度所占的位数(bit)。若每个像素具有8位的颜色深度，则可表示（13）种不同的颜色。

- A. 8
- B. 64
- C. 256
- D. 512

第14题：视觉上的颜色可用亮度、色调和饱和度三个特征来描述。其中饱和度是指颜色的（14）。

- A. 种数
- B. 纯度
- C. 感觉
- D. 存储量

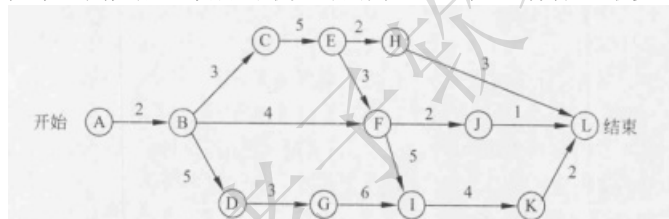
第15题：若用户需求不清晰且经常发生变化，但系统规模不太大且不太复杂，则最适宜采用（15）开发方法，对于数据处理领域的问题，若系统规模不太大且不太复杂，需求变化也不大，则最适宜采用（16）开发方法。

- A. 结构化
- B. Jackson
- C. 原型化
- D. 面向对象

第16题：若用户需求不清晰且经常发生变化，但系统规模不太大且不太复杂，则最适宜采用（15）开发方法，对于数据处理领域的问题，若系统规模不太大且不太复杂，需求变化也不大，则最适宜采用（16）开发方法。

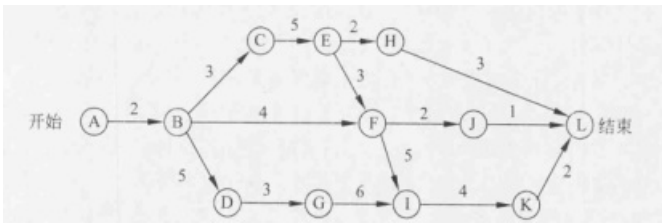
- A. 结构化
- B. Jackson
- C. 原型化
- D. 面向对象

第17题：某软件项目的活动图如下图所示，其中顶点表示项目里程碑，连接顶点的边表示活动，边上的数字表示该活动所需的天数，则完成该项目的最少时间为（17）天。活动BD最多可以晚（18）天开始而不会影响整个项目的进度。



- A. 9
- B. 15
- C. 22
- D. 24

第18题：某软件项目的活动图如下图所示，其中顶点表示项目里程碑，连接顶点的边表示活动，边上的数字表示该活动所需的天数，则完成该项目的最少时间为（17）天。活动BD最多可以晚（18）天开始而不会影响整个项目的进度。



- A. 2
- B. 3
- C. 5
- D. 9

第19题：以下关于软件项目管理中人员管理的叙述，正确的是（19）。

- A. 项目组成员的工作风格也应该作为组织团队时要考虑的一个要素
- B. 鼓励团队的每个成员充分地参与开发过程的所有阶段
- C. 仅根据开发人员的能力来组织开发团队
- D. 若项目进度滞后于计划，则增加开发人员一定可以加快开发进度

第20题：编译器和解释器是两种基本的高级语言处理程序。编译器对高级语言源程序的处理过程可以划分为词法分析、语法分析、语义分析、中间代码生成、代码优化、目标代码生成等阶段，其中，（20）并不是每个编译器都必需的，与编译器相比，解释器（21）。

- A. 词法分析和语法分析
- B. 语义分析和中间代码生成
- C. 中间代码生成和代码优化
- D. 代码优化和目标代码生成

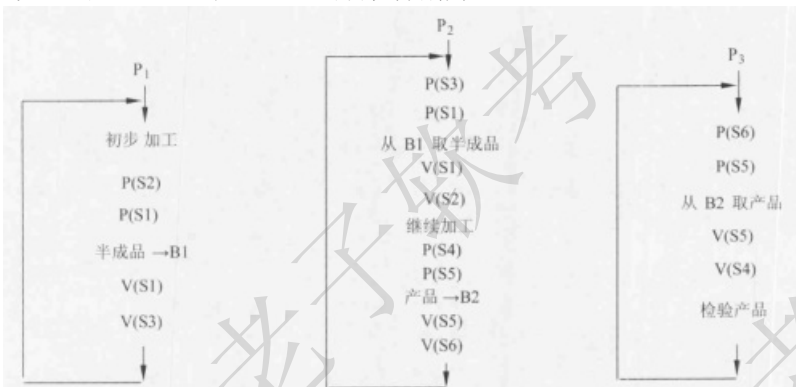
第21题：编译器和解释器是两种基本的高级语言处理程序。编译器对高级语言源程序的处理过程可以划分为词法分析、语法分析、语义分析、中间代码生成、代码优化、目标代码生成等阶段，其中，（20）并不是每个编译器都必需的，与编译器相比，解释器（21）。

- A. 不参与运行控制，程序执行的速度慢
- B. 参与运行控制，程序执行的速度慢
- C. 参与运行控制，程序执行的速度快
- D. 不参与运行控制，程序执行的速度快

第22题：表达式采用逆波兰式表示时，利用（22）进行求值。

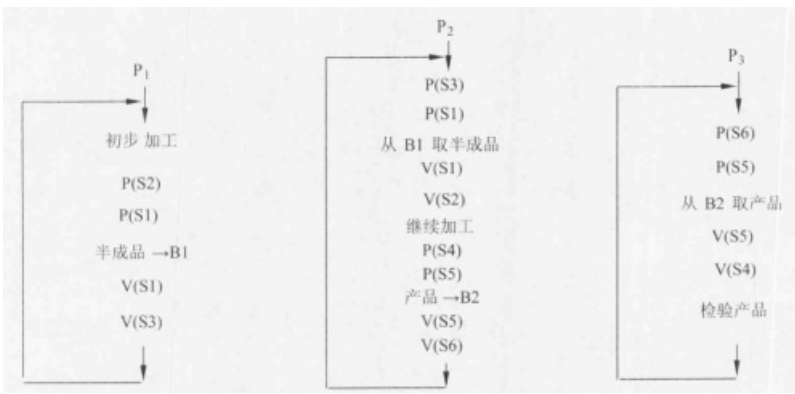
- A. 栈
- B. 队列
- C. 符号表
- D. 散列表

第23题：某企业的生产流水线上有2名工人P1和P2，1名检验员P3。P1将初步加工的半成品放入半成品箱B1；P2从半成品箱B1取出继续加工，加工好的产品放入成品箱B2；P3从成品箱B2去除产品校验。假设B1可存放n件半成品，B2可存放m件产品，并设置6个信号量S1、S2、S3、S4、S5和S6，且S3和S6的初值都为0。采用PV操作实现P1、P2和P3的同步模型如下图所示，则信号量S1和S5（23）；S2、S4的初值分别为（24）。



- A. 分别为同步信号量和互斥信号量，初值分别为0和1
- B. 都是同步信号量，其初值分别为0和0
- C. 都是互斥信号量，其初值分别为1和1
- D. 都是互斥信号量，其初值分别为0和1

第24题：某企业的生产流水线上有2名工人P1和P2，1名检验员P3。P1将初步加工的半成品放入半成品箱B1；P2从半成品箱B1取出继续加工，加工好的产品放入成品箱B2；P3从成品箱B2去除产品校验。假设B1可存放n件半成品，B2可存放m件产品，并设置6个信号量S1、S2、S3、S4、S5和S6，且S3和S6的初值都为0。采用PV操作实现P1、P2和P3的同步模型如下图所示，则信号量S1和S5（23）；S2、S4的初值分别为（24）。



- A. n、0
- B. m、0
- C. m、n
- D. n、m

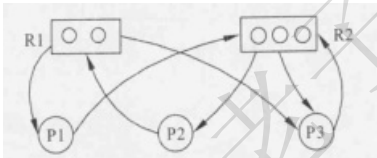
第25题：假设磁盘块与缓冲区大小相同，每个盘块读入缓冲区的时间为 $15\mu s$ ，由缓冲区送至用户区的时间是 $5\mu s$ ，在用户区内系统对每块数据的处理时间为 $1\mu s$ ，若用户需要将大小为10个磁盘块的Doc1文件逐块从磁盘读入缓冲区，并送至用户区进行处理，那么采用单缓冲区需要花费的时间为（25） μs ；采用双缓冲区需要花费的时间为（26） μs 。

- A. 150
- B. 151
- C. 156
- D. 201

第26题：假设磁盘块与缓冲区大小相同，每个盘块读入缓冲区的时间为 $15\mu s$ ，由缓冲区送至用户区的时间是 $5\mu s$ ，在用户区内系统对每块数据的处理时间为 $1\mu s$ ，若用户需要将大小为10个磁盘块的Doc1文件逐块从磁盘读入缓冲区，并送至用户区进行处理，那么采用单缓冲区需要花费的时间为（25） μs ；采用双缓冲区需要花费的时间为（26） μs 。

- A. 150
- B. 151
- C. 156
- D. 201

第27题：在如下所示的进程资源图中，（27）。



- A. P1、P2、P3都是非阻塞节点，该图可以化简，所以是非死锁的
- B. P1、P2、P3都是阻塞节点，该图不可以化简，所以是死锁的
- C. P1、P2是非阻塞节点，P3是阻塞节点，该图不可以化简，所以是死锁的
- D. P2是阻塞节点，P1、P3是非阻塞节点，该图可以化简，所以是非死锁的

第28题：在支持多线程的操作系统中，假设进程P创建了若干个线程，那么（28）是不能被这些线程共享的。

- A. 该进程中打开的文件
- B. 该进程的代码段
- C. 该进程中某线程的栈指针
- D. 该进程的全局变量

第29题：某开发小组欲开发一个超大规模软件：使用通信卫星，在订阅者中提供、监视和控制移动电话通信，则最不宜采用（29）过程模型。

- A. 瀑布
- B. 原型
- C. 螺旋
- D. 喷泉

第30题：（30）开发过程模型以用户需求为动力，以对象为驱动，适合于面向对象的开发方法。

- A. 瀑布
- B. 原型
- C. 螺旋
- D. 喷泉

第31题：在ISO/IEC软件质量模型中，易使用性的子特性不包括（31）。

- A. 易理解性

- B. 易学性
- C. 易操作性
- D. 易分析性

第32题：在进行子系统结构设计时，需要确定划分后的子系统模块结构，并画出模块结构图。该过程不需要考虑（32）。

- A. 每个子系统如何划分成多个模块
- B. 每个子系统采用何种数据结构和核心算法
- C. 如何确定子系统之间、模块之间传送的数据及其调用关系
- D. 如何评价并改进模块结构的质量

第33题：数据流图中某个加工的一组动作依赖于多个逻辑条件的取值，则用（33）能够清楚地表示复杂的条件组合与应做的动作之间的对应关系。

- A. 流程图
- B. NS盒图
- C. 形式语言
- D. 决策树

第34题：根据软件过程活动对软件工具进行分类，则逆向工程工具属于（34）工具。

- A. 软件开发
- B. 软件维护
- C. 软件管理
- D. 软件支持

第35题：若用白盒测试方法测试以下代码，并满足条件覆盖，则至少需要（35）个测试用例。采用McCabe度量法算出该程序的环路复杂性为（36）。

```
int find_max(int i, int j, int k){
    int max;
    if (i > j) then
        if (i > k) then max = i;
        else max = k;
    else if (j > k) max = j;
    else max = k;
    return max;
}
```

- A. 3
- B. 4
- C. 5
- D. 6

第36题：若用白盒测试方法测试以下代码，并满足条件覆盖，则至少需要（35）个测试用例。采用McCabe度量法算出该程序的环路复杂性为（36）。

```
int find_max(int i, int j, int k){
    int max;
    if (i > j) then
        if (i > k) then max = i;
        else max = k;
    else if (j > k) max = j;
    else max = k;
    return max;
}
```

- A. 1
- B. 2
- C. 3
- D. 4

第37题：在面向对象的系统中，对象是运行时实体，其组成部分不包括（37）；一个类定义了一组大体相似的对象，这些对象共享（38）。

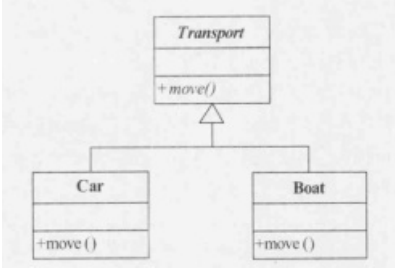
- A. 消息
- B. 行为（操作）
- C. 对象名
- D. 状态

第38题：在面向对象的系统中，对象是运行时实体，其组成部分不包括（37）；一个类定义了一组大体相似的对象，这些对象共享（38）。

- A. 属性和状态
- B. 对象名和状态
- C. 行为和多重度

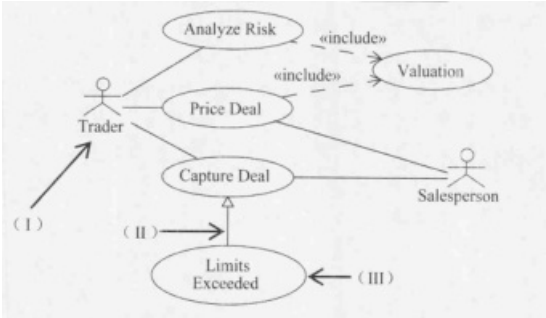
D. 属性和行为

第39题：如下所示的UML类图中，Car和Boat类中的move()方法（39）了Transport类中的move()方法。



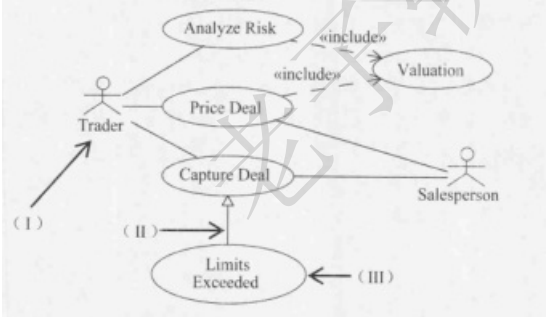
- A. 继承
- B. 覆盖（重置）
- C. 重载
- D. 聚合

第40题：如下所示的UML图中，(I)是（40），(II)是（41），(III)是（42）。



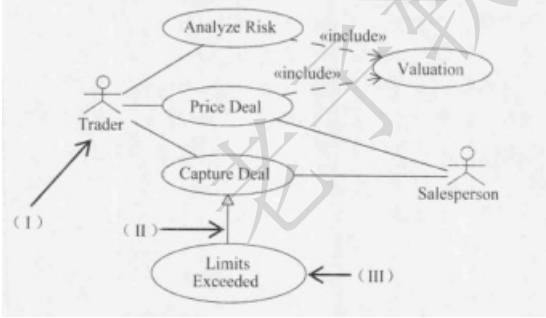
- A. 参与者
- B. 用例
- C. 泛化关系
- D. 包含关系

第41题：如下所示的UML图中，(I)是（40），(II)是（41），(III)是（42）。



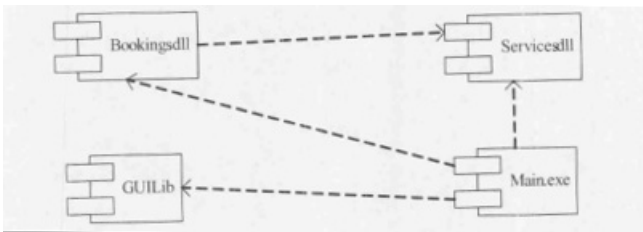
- A. 参与者
- B. 用例
- C. 泛化关系
- D. 包含关系

第42题：如下所示的UML图中，(I)是（40），(II)是（41），(III)是（42）。



- A. 参与者
- B. 用例
- C. 泛化关系
- D. 包含关系

第43题：下所示为UML（43）。



- A. 类图
- B. 部署图
- C. 组件图
- D. 网络图

第44题：以下关于Singleton（单例）设计模式的叙述中，不正确的是（44）。

- A. 单例模式是创建型模式
- B. 单例模式保证一个类仅有一个实例
- C. 单例类提供一个访问唯一实例的全局访问点
- D. 单例类提供一个创建一系列相关或相互依赖对象的接口

第45题：（45）设计模式能够动态地给一个对象添加一些额外的职责而无需修改此对象的结构；（46）设计模式定义一个用于创建对象的接口，让子类决定实例化哪一个类；欲使一个后端数据模型能够被多个前端用户界面连接，采用（47）模式最适合。

- A. 组合（Composite）
- B. 外观(Facade)
- C. 享元（Flyweight）
- D. 装饰器(Decorator)

第46题：（45）设计模式能够动态地给一个对象添加一些额外的职责而无需修改此对象的结构；（46）设计模式定义一个用于创建对象的接口，让子类决定实例化哪一个类；欲使一个后端数据模型能够被多个前端用户界面连接，采用（47）模式最适合。

- A. 工厂方法（Factory Method）
- B. 享元（Flyweight）
- C. 观察者（Observer）
- D. 中介者(Mediator)

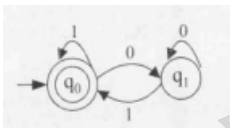
第47题：（45）设计模式能够动态地给一个对象添加一些额外的职责而无需修改此对象的结构；（46）设计模式定义一个用于创建对象的接口，让子类决定实例化哪一个类；欲使一个后端数据模型能够被多个前端用户界面连接，采用（47）模式最适合。

- A. 装饰器(Decorator)
- B. 享元（Flyweight）
- C. 观察者(Observer)
- D. 中介者(Mediator)

第48题：某程序运行时陷入死循环，则可能的原因是程序中存在（48）。

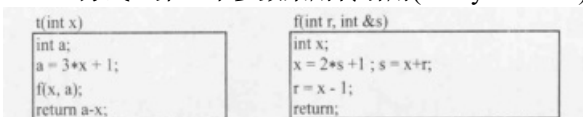
- A. 词法错误
- B. 语法错误
- C. 动态的语义错误
- D. 静态的语义错误

第49题：某非确定的有限自动机(NFA)的状态转换图如下图所示（q₀既是初态也是终态）。以下关于该NFA的叙述中，正确的是（49）。



- A. 其可识别的0、1序列的长度为偶数
- B. 其可识别的0、1序列中0与1的个数相同
- C. 其可识别的非空0、1序列中开头和结尾字符都是0
- D. 其可识别的非空0、1序列中结尾字符是1

第50题：函数t()、f()的定义如下所示，若调用函数t时传递给x的值为5，并且调用函数F()时，第一个参数采用传值（call by value）方式，第二个参数采用传引用(call by reference)方式，则函数t的返回值为（50）。



- A. 33
- B. 22
- C. 11
- D. 负数

第51题：数据库系统通常采用三级模式结构：外模式、模式和内模式。这三级模式分别对应数据库的（51）。

- A. 基本表、存储文件和视图
- B. 视图、基本表和存储文件
- C. 基本表、视图和存储文件
- D. 视图、存储文件和基本表

第52题：在数据库逻辑设计阶段，若实体中存在多值属性，那么将E-R图转换为关系模式时，（52），得到的关系模式属于4NF。

- A. 将所有多值属性组成一个关系模式
- B. 使多值属性不在关系模式中出现
- C. 将实体的码分别和每个多值属性独立构成一个关系模式
- D. 将多值属性和其它属性一起构成该实体对应的关系模式

第53题：在分布式数据库中有分片透明、复制透明、位置透明和逻辑透明等基本概念，其中：（53）是指局部数据模型透明，即用户或应用程序无需知道局部使用的是哪种数据模型；（54）是指用户或应用程序不需要知道逻辑上访问的表具体是如何分块存储的。

- A. 分片透明
- B. 复制透明
- C. 位置透明
- D. 逻辑透明

第54题：在分布式数据库中有分片透明、复制透明、位置透明和逻辑透明等基本概念，其中：（53）是指局部数据模型透明，即用户或应用程序无需知道局部使用的是哪种数据模型；（54）是指用户或应用程序不需要知道逻辑上访问的表具体是如何分块存储的。

- A. 分片透明
- B. 复制透明
- C. 位置透明
- D. 逻辑透明

第55题：设有关系模式R(A1,A2,A3,A4,A5,A6)，其中：函数依赖集F={A1→A2,A1A3→A4,A5A6→A1,A2A5→A6,A3A5→A6}，则（55）是关系模式R的一个主键，R规范化程度最高达到（56）。

- A. A1A4
- B. A2A4
- C. A3A5
- D. A4A5

第56题：设有关系模式R(A1,A2,A3,A4,A5,A6)，其中：函数依赖集F={A1→A2,A1A3→A4,A5A6→A1,A2A5→A6,A3A5→A6}，则（55）是关系模式R的一个主键，R规范化程度最高达到（56）。

- A. 1NF
- B. 2NF
- C. 3NF
- D. BCNF

第57题：对于一个长度为n(n>1)且元素互异的序列，每其所有元素依次通过一个初始为空的栈后，再通过一个初始为空的队列。假设队列和栈的容量都足够大，且只要栈非空就可以进行出栈操作，只要队列非空就可以进行出队操作，那么以下叙述中，正确的是（57）。

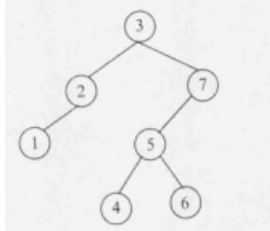
- A. 出队序列和出栈序一定互为逆序
- B. 出队序列和出栈序列一定相同
- C. 入栈序列与入队序列一定相同
- D. 入栈序列与入队序列一定互为逆序

第58题：设某n阶三对角矩阵 $A_{n \times n}$ 的示意图如下图所示。若将该三对角矩阵的非零元素按行存储在一维数组B[k] ($1 \leq k \leq 3 \cdot n - 2$)中，则k与i、j的对应关系是（58）。

$$A_{n \times n} = \begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ & a_{2,1} & a_{2,2} & a_{2,3} & & 0 \\ & & a_{3,2} & a_{3,3} & a_{3,4} & \\ & & & \dots & \dots & \dots \\ 0 & & & & \dots & \dots & \dots \\ & & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

- A. $k=2i+j-2$
- B. $k=2i-j+2$
- C. $k=3i+j-1$
- D. $k=3i-j+2$

第59题：对于非空的二叉树，设D代表根结点，L代表根结点的左子树R代表根结点的右子树。若对下图所示的二叉树进行遍历后的结点序列为7 6 5 4 3 2 1，则遍历方式是（59）。



- A. LRD
- B. DRL
- C. RLD
- D. RDL

第60题：在55个互异元素构成的有序表A[1..55]中进行折半查找（或二分查找，向下取整）。若需要找的元素等于A[19]，则在查找过程中参与比较的元素依次为（60）、A[19]。

- A. A[28]、A[30]、A[15]、A[20]
- B. A[28]、A[14]、A[21]、A[17]
- C. A[28]、A[15]、A[22]、A[18]
- D. A[28]、A[18]、A[22]、A[20]

第61题：设一个包含n个顶点、e条弧的简单有向图采用邻接矩阵存储结构（即矩阵元素A[i][j]等于1或0，分别表示顶点i与顶点j之间有弧或无弧），则该矩阵的非零元素数目为（61）。

- A. e
- B. 2e
- C. n-e
- D. n+e

第62题：已知算法A的运行时间函数为 $T(n)=8T(n/2)+n^2$ ，其中n表示问题的规模，则该算法的时间复杂度为（62）。另已知算法B的运行时间函数为 $T(n)=XT(n/4)+n^2$ ，其中n表示问题的规模。对充分大的n，若要算法B比算法A快，则X的最大值为（63）。

- A. $\theta(n)$
- B. $\theta(n \lg n)$
- C. $\theta(n^2)$
- D. $\theta(n^3)$

第63题：已知算法A的运行时间函数为 $T(n)=8T(n/2)+n^2$ ，其中n表示问题的规模，则该算法的时间复杂度为（62）。另已知算法B的运行时间函数为 $T(n)=XT(n/4)+n^2$ ，其中n表示问题的规模。对充分大的n，若要算法B比算法A快，则X的最大值为（63）。

- A. 15
- B. 17
- C. 63
- D. 65

第64题：在某应用中，需要先排序一组大规模的记录，其关键字为整数。若这组记录的关键字基本上有序，则适宜采用（64）排序算法。若这组记录的关键字的取值均在0到9之间（含），则适宜采用（65）排序算法。

- A. 插入
- B. 归并
- C. 快速
- D. 计数

第65题：在某应用中，需要先排序一组大规模的记录，其关键字为整数。若这组记录的关键字基本上有序，则适宜采用（64）

排序算法。若这组记录的关键字的取值均在0到9之间（含），则适宜采用（65）排序算法。

- A. 插入
- B. 归并
- C. 快速
- D. 基数

第66题：集线器与网桥的区别是：（66）。

- A. 集线器不能检测发送冲突，而网桥可以检测冲突
- B. 集线器是物理层设备，而网桥是数据链路层设备
- C. 网桥只有两个端口，而集线器是一种多端口网桥
- D. 网桥是物理层设备，而集线器是数据链路层设备

第67题：POP3协议采用（67）模式，客户端代理与POP3服务器通过建立TCP连接来传送数据。

- A. Browser/Server
- B. Client/Server
- C. Peer to Peer
- D. Peer to Server

第68题：TCP使用的流量控制协议是（68）。

- A. 固定大小的滑动窗口协议
- B. 后退N帧的ARQ协议
- C. 可变大小的滑动窗口协议
- D. 停等协议

第69题：以下4种路由中，（69）路由的子网掩码是255.255.255.255。

- A. 远程网络
- B. 静态
- C. 默认
- D. 主机

第70题：以下关于层次化局域网模型中核心层的叙述，正确的是（70）。

- A. 为了保障安全性，对分组要进行有效性检查
- B. 将分组从一个区域高速地转发到另一个区域
- C. 由多台二、三层交换机组成
- D. 提供多条路径来缓解通信瓶颈

第71题：In a world where it seems we already have too much to do, and too many things to think about, it seems the last thing we need is something new that we have to learn.

But use cases do solve a problem with requirements: with (71) declarative requirements it's hard to describe steps and sequences of events. Use cases, stated simply, allow description of sequences of events that, taken together, lead to a system doing something useful. As simple as this sounds, this is important. When confronted only with a pile of requirements, it's often (72) to make sense of what the authors of the requirements really wanted the system to do. In the preceding example, use cases reduce the ambiguity of the requirements by specifying exactly when and under what conditions certain behavior occurs; as such, the sequence of the behaviors can be regarded as a requirement. Use cases are particularly well suited to capture approaches. Although this may sound simple, the fact is that (73) requirement capture approaches, with their emphasis on declarative requirements and "shall" statements, completely fail to capture the (74) of the system's behavior. Use cases are a simple yet powerful way to express the behavior of the system in way that all stakeholders can easily understand. But, like anything, use cases come with their own problems, and as useful as they are, they can be (75). The result is something that is as bad, if not worse, than the original problem. Therein it's important to utilize use cases effectively without creating a greater problem than the one you started with.

- A. plenty
- B. loose
- C. extra
- D. strict

第72题：In a world where it seems we already have too much to do, and too many things to think about, it seems the last thing we need is something new that we have to learn.

But use cases do solve a problem with requirements: with (71) declarative requirements it's hard to describe steps and sequences of events. Use cases, stated simply, allow description of sequences of events that, taken together, lead to a system doing something useful. As simple as this sounds, this is important. When confronted only with a pile of requirements, it's often (72) to make sense of what the authors of the requirements really wanted the system to do. In the preceding example, use cases reduce the ambiguity of the requirements by specifying exactly when and under what conditions certain behavior occurs; as such, the sequence of the behaviors can be regarded as a requirement. Use cases are particularly well suited to capture approaches. Although this may sound simple, the fact is that (73) requirement capture approaches, with their emphasis on declarative requirements and "shall" statements, completely fail to capture the (74) of the system's behavior. Use cases are a simple yet powerful way to express the behavior of the system in way that all stakeholders can easily understand. But, like anything, use cases come with their own problems, and as useful as they are, they can be (75). The result is something that is as bad, if not worse, than the original problem. Therein it's important to utilize use cases effectively without creating a greater problem than the one you started with.

not worse, that the original problem. Therein it's important to utilize use cases effectively without creating a greater problem than the one you started with.

- A. impossible
- B. possible
- C. sensible
- D. practical

第73题: In a world where it seems we already have too much to do, and too many things to think about, it seems the last thing we need is something new that we have to learn.

But use cases do solve a problem with requirements: with (71) declarative requirements it's hard to describe steps and sequences of events. Use cases, stated simply, allow description of sequences of events that, taken together, lead to a system doing something useful. As simple as this sounds, this is important. When confronted only with a pile of requirements, it's often (72) to make sense of what the authors of the requirements really wanted the system to do. In the preceding example, use cases reduce the ambiguity of the requirements by specifying exactly when and under what conditions certain behavior occurs; as such, the sequence of the behaviors can be regarded as a requirement. Use cases are particularly well suited to capture approaches. Although this may sound simple, the fact is that (73) requirement capture approaches, with their emphasis on declarative requirements and "shall" statements, completely fail to capture fail to capture the (74) of the system's behavior. Use cases are a simple yet powerful way to express the behavior of the system in way that all stakeholders can easily understand.

But, like anything, use cases come with their own problems, and as useful as they are, they can be (75). The result is something that is as bad, if not worse, than the original problem. Therein it's important to utilize use cases effectively without creating a greater problem than the one you started with.

- A. modern
- B. conventional
- C. different
- D. formal

第74题: In a world where it seems we already have too much to do, and too many things to think about, it seems the last thing we need is something new that we have to learn.

But use cases do solve a problem with requirements: with (71) declarative requirements it's hard to describe steps and sequences of events. Use cases, stated simply, allow description of sequences of events that, taken together, lead to a system doing something useful. As simple as this sounds, this is important. When confronted only with a pile of requirements, it's often (72) to make sense of what the authors of the requirements really wanted the system to do. In the preceding example, use cases reduce the ambiguity of the requirements by specifying exactly when and under what conditions certain behavior occurs; as such, the sequence of the behaviors can be regarded as a requirement. Use cases are particularly well suited to capture approaches. Although this may sound simple, the fact is that (73) requirement capture approaches, with their emphasis on declarative requirements and "shall" statements, completely fail to capture fail to capture the (74) of the system's behavior. Use cases are a simple yet powerful way to express the behavior of the system in way that all stakeholders can easily understand.

But, like anything, use cases come with their own problems, and as useful as they are, they can be (75). The result is something that is as bad, if not worse, than the original problem. Therein it's important to utilize use cases effectively without creating a greater problem than the one you started with.

- A. statics
- B. nature
- C. dynamics
- D. originals

第75题: In a world where it seems we already have too much to do, and too many things to think about, it seems the last thing we need is something new that we have to learn.

But use cases do solve a problem with requirements: with (71) declarative requirements it's hard to describe steps and sequences of events. Use cases, stated simply, allow description of sequences of events that, taken together, lead to a system doing something useful. As simple as this sounds, this is important. When confronted only with a pile of requirements, it's often (72) to make sense of what the authors of the requirements really wanted the system to do. In the preceding example, use cases reduce the ambiguity of the requirements by specifying exactly when and under what conditions certain behavior occurs; as such, the sequence of the behaviors can be regarded as a requirement. Use cases are particularly well suited to capture approaches. Although this may sound simple, the fact is that (73) requirement capture approaches, with their emphasis on declarative requirements and "shall" statements, completely fail to capture fail to capture the (74) of the system's behavior. Use cases are a simple yet powerful way to express the behavior of the system in way that all stakeholders can easily understand.

But, like anything, use cases come with their own problems, and as useful as they are, they can be (75). The result is something that is as bad, if not worse, than the original problem. Therein it's important to utilize use cases effectively without creating a greater problem than the one you started with.

- A. misapplied
- B. applied
- C. used
- D. powerful